MM/GBSA tutorials for SARS-CoV-2 Mpro in complex with inhibitor N3 by MolAICal and NAMD

Qifeng Bai
Email: molaical@yeah.net
Homepage: https://molaical.github.io or https://molaical.gitlab.io
School of Basic Medical Sciences
Lanzhou University
Lanzhou, Gansu 730000, P. R. China

1. Introduction

In this tutorial, the MolAICal (https://molaical.github.io) is used to calculate the MM/GBSA between ligand N3 and SARS-CoV-2 Mpro based on molecular dynamical (MD) simulated results by NAMD. This tutorial is just a demo. To save running and storage space, only 25 frames of MD simulated trajectories of SARS-CoV-2 Mpro in complex with N3 are selected for this tutorial. This tutorial can **not only** be used to calculate MM/GBSA of protein-ligand **but also** MM/GBSA of protein-peptide, protein-protein, DNA-ligand, DNA-protein, protein-DNA, protein-RNA, and any complex based on the run MD simulations. It just replaces the protein and ligand of this tutorial with the appointed objects. For example, **protein** in this tutorial is substituted for **DNA**, and **ligand** in this tutorial is substituted for **peptide** **based on the same running command arguments of this tutorial**.

2. Materials

2.1. Software requirement

1) MolAICal: https://molaical.github.io

2) NAMD: https://www.ks.uiuc.edu/Research/namd/

(**Notice:** This tutorial can be run by **NAMD 2.x, 3.x, or a higher version.** For example, the command "**namd3**" substitutes for the command "**namd2**" in this tutorial **if you use NAMD 3.x version**. For a higher version of NAMD, users can use a similar replacement way as the previous example.)

3) Carma: https://github.com/glykos/carma or https://utopia.duth.gr/~glykos/Carma.html

2.2. Example files

1) All the necessary tutorial files are downloaded from:
https://github.com/MolAICal/tutorials/tree/master/004-MMGBSA

3. Procedure

Part I. Problem solution

Problem: Some users meet the positive value problem when calculating MM/GBSA:
https://www.ks.uiuc.edu/Research/namd/mailing_list/namd-l.2020-2021/1295.html

Issues: ΔG binding is positive. During the MD simulations, one independent molecule may go into another image. So it seems no interactions with the other molecule if it only depends on distances. Actually, they still have the interaction because of periodic boundary conditions. If the users' research subject does not involve multiple molecules entering different images, **this section can be skipped, and** users may proceed directly **to Part II.**

Possible solutions: it must check whether the ligand is always in the same periodic boundary with the receptor along the entire simulated time. If not, please use the VMD PBC to wrap the ligand into the receptor along simulated time. More detailed:

https://www.ks.uiuc.edu/Research/vmd/script_library/scripts/pbcwrap/
https://www.ks.uiuc.edu/Research/vmd/plugins/pbctools/

I supply two commands to run on the VMD Tk Console:
%> pbc wrap -centersel protein -center com -compound chain -all

Note: checking the complex in VMD, if the above command can wrap the complex together. It can skip this command below (pbc unwrap):

%> pbc unwrap -sel "not (water or ions)" -all

Running above one or two commands can wrap the ligand into the receptor over the simulated time. Saving wrap or unwrap trajectories into a DCD file using the below command:
%> set all [atomselect top all]
%> animate write dcd mpro.dcd sel $all beg 0 end 24 waitfor all

Here, all atoms are selected. Users can select the wanted part. Our tutorial trajectories only have 25 frames. So the beginning is 0 and the end is 24. "mpro.dcd" is just a name. And then starting the following steps.

Option (Wrapping operation without GUI)

If users have wrapped all complex or have no requirement for the DCD wrapping operation, or VMD and NAMD have already been configured within the Linux version of the MolAICal container, users can skip this option directly.

Users may execute and test the aforementioned commands within either Windows or Linux graphical desktop environments. However, **when operating in environments lacking graphical interfaces, command-line input becomes necessary.** To facilitate external program execution, MolAICal provides a persistent configuration interface. The following command establishes the name and path for VMD (Visual Molecular Dynamics), **requiring only a single configuration that remains available for subsequent use without repetition**.

To address the library dependency issues in Linux, the Linux version of MolAICal **(Not for Windows version)** adopts a container-based approach. If **external programs need** to be invoked, it is **recommended** to install them **within** the MolAICal **container**. If **no external** programs are required, this step can be **ignored**. This tutorial requires the external programs NAMD and VMD, and the steps are as follows:

a) First of all, copy the files to the MolAICal container.

```
****** 1.  Go to the container file system, it will enter the "/root" ******
#> molaical.exe -eset shell in
```

```
****** 2.  The first part of 'cp' is from the local machine, and the second part is in the container; ******
****** The VMD and NAMD packages can be moved into the container by the command below ******
#> cp /home/user/<local machine file> /root/soft

****** 3.  Exit container file system ******
#> exit
```

b) Secondly, enter the container virtual environment of MolAICal:

```
#> molaical.exe -eset sys run molaical
```

Note: 'molaical' is the container name.

c) Installing software in the container virtual environment of MolAICal is the same as installing it on the host local machine. Here, we take the installation of VMD and NAMD as examples:

1) For NAMD:

Extract the NAMD files, assuming the extracted folder is named namdcpu. Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n NAMD -p "/root/soft/namdcpu/namd3"
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system. The string "VMD" and "NAMD" (case insensitive) after "-n" is fixed for the paths of VMD and NAMD. To ensure the reproducibility of MM/GBSA results, it is recommended to use the CPU version of NAMD, as the "seed" parameter appears to be ineffective for reproducibility in the CUDA version of NAMD.

2) For VMD:

Following the steps below:

✧ Extract the VMD files:

```
#> tar -xzvf vmd-xxx.tar.gz
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system.

✧ Modify the install path in a file named "**configure**" in the decompressed folder of VMD:

```
The default: $install_bin_dir="/usr/local/bin"; modify it to →
$install_bin_dir="/root/soft/vmd193/bin";

The default: $install_library_dir="/usr/local/lib/$install_name"; modify it to →
$install_library_dir="/root/soft/vmd193/lib/$install_name";
```

✧ Install VMD:

```
#> cd vmd-xx
#> ./configure LINUXAMD64
#> cd src
#> make install
```

Note: Please run **./configure**, and select the right types for the used computers. Here, it is "LINUXAMD64".

✧ Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n VMD -p "/root/soft/vmd193/bin/vmd"
```

So far, all the installations and configurations of NAMD and VMD are complete in the MolAICal container.

To prevent the loss of installed programs (e.g., VMD and NAMD) when rebuilding the MolAICal image, refer to step 3 in **Appendix 2**.

Then, run the arguments of the external program:

```
#> molaical.exe -call run -n vmd -i -dispdev text -e pbcwrap.vmd
```

Note:

1) In MolAICal's environment variable configuration, the parameter name following '-n' (e.g., '-n VMD') must match the corresponding runtime parameter (e.g., '-n vmd') in terms of spelling, though case sensitivity is not enforced. As shown in the two commands above, the parameter names following '-n' must be consistent.
2) The '-i' flag is followed by VMD runtime arguments. '-i' must be placed after other arguments. Here, simply replace pbcwrap.vmd with the users' own script file, though users may optionally use this tutorial-provided script pbcwrap.vmd.

"pbcwrap.vmd" is shown as an example below (it can be modified according to the purpose or assay results of users):

```
package require pbctools
mol new mpro.psf
mol addfile mpro.dcd waitfor all
pbc wrap -centersel protein -center com -compound chain -all
set all [atomselect top all]
animate write dcd mpro.dcd sel $all beg 0 end 24 waitfor all
quit
```

Part II. MM/GBSA calculation based on NAMD

Assuming users have configured the internal commands in MolAICal for VMD and NAMD using the following instructions:

```
#> molaical.exe -call set -n VMD -p "C:\Program Files (x86)\University of Illinois\VMD\vmd.exe"
#> molaical.exe -call set -n namd -p "d:\namd2\namd2.exe"
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system. Please note that the configuration of MolAICal differs between the Windows and Linux versions: the Linux version utilizes the udocker container approach and requires setup within the

container, whereas the Windows version does not.

3.1. MM/GBSA calculation

Firstly, I assume the equilibrium trajectory named "mpro.dcd" has been run by NAMD. Receptor and ligand have been put into the same periodic boundary in "mpro.dcd". Users can replace "mpro.dcd" with their own trajectory. Go to the tutorial directory:
#> cd 004-MMGBSA

3.1.1. Extracting trajectory of protein in complex with ligand

#> molaical.exe -call run -n vmd -c stripDCD -i -dispdev text -psf "mpro.psf" -args protein,or,resname,LIG "mpro.dcd" "complex" mpro.psf mpro.pdb

-i:	after "-i", follow with VMD-related commands. The command-line argument "-i" **must** be placed **after** other arguments (e.g., after argument "-c").
-args:	it is the usage like the command "**atomselect**" in the Tk Console of VMD software, such as "atomselect top protein or resname LIG". Here, the comma "," represents a blank space " ".
-call:	when its value is "run", it will call the external program.
-n:	the external program name, it will automatically search the path of the program based on the name of this program from the inner environment variables of MolAICal.
-c:	the calculation way, it will be used to deal with the DCD file if its value is "stripDCD".

"complex" is the prefix of the output files. The mpro.psf and mpro.pdb are referred to generate the files that have the prefix "complex". It will generate complex.psf, complex.pdb, and complex.dcd. Turning on the parameters of "GBIS" and "sasa". Open "complex.conf" and modify the appropriate parameters of red fonts as below:

---

structure complex.psf
coordinates complex.pdb
outputName complex

paraTypeCharmm on
parameters par_all36_prot.prm
parameters par_all36_cgenff.prm
parameters ligand.str
parameters toppar_water_ions.str

coorfile open dcd complex.dcd

Our tutorial is run by CPU. Users can run it on GPUs. Running NAMD command in Linux operating system as below:
#> molaical.exe -call run -n namd -i +p1 complex.conf >& complex.log &

Where the symbol “&” assigns the command to run in the background on the Linux operating system. If NAMD runs on the Windows operating system, the symbol “&” must be omitted. For instance:

#> molaical.exe -call run -n namd -i +p1 complex.conf > complex.log

```
##############################################################################
# Notice: This tutorial can be run by NAMD 2.x, 3.x, or a higher version. For example, the   #
# command "namd3" substitutes for the command "namd2" in this tutorial if you use NAMD      #
# 3.x version. For a higher version of NAMD, users can use a similar replacement way as the #
# previous example. In this tutorial, the command "namd2" is used.                          #
##############################################################################
```

3.1.2. Extracting trajectory of protein only.

#> molaical.exe -call run -n vmd -c stripDCD -i -dispdev text -psf "mpro.psf" -args protein "mpro.dcd" "protein" mpro.psf mpro.pdb

"protein" is the prefix of the output files. The mpro.psf and mpro.pdb are referred to generate the files that have the prefix "protein". It will generate protein.psf, protein.pdb, and protein.dcd. Open “protein.conf” and modify the appropriate parameters in a similar way to “complex.conf”.

Our tutorial is run by CPU. Users can run it on GPUs. Running NAMD command in Linux operating system as below:

#> molaical.exe -call run -n namd -i +p1 protein.conf >& protein.log &

3.1.3. Extracting trajectory of ligand only.

#> molaical.exe -call run -n vmd -c stripdcd -i -dispdev text -psf "mpro.psf" -args resname,LIG "mpro.dcd" "ligand" mpro.psf mpro.pdb

"ligand" is the prefix of the output files. The mpro.psf and mpro.pdb are referred to generate the files that have the prefix "ligand". It will generate ligand.psf, ligand.pdb, and ligand.dcd. Open “ligand.conf” and modify the appropriate parameters in a similar way to “complex.conf”.

Our tutorial is run by CPU. You can run it on GPU. Running NAMD command in Linux operating system as below:

#> molaical.exe -call run -n namd -i +p1 ligand.conf >& ligand.log &

3.1.4. Calculating MM/GBSA by MolAICal

#> molaical.exe -mmgbsa -c complex.log -r protein.log -l ligand.log

The output contains the binding free energy △G as below:

```
---------------------------------------------------------------------
delta E(internal): -0
delta E(electrostatic) + deltaG(sol): 9.0625
delta E(VDW): -52.1546
delta G binding: -43.0921    +/-    1.5748 (kcal/mol)
---------------------------------------------------------------------
```

Note: Results may vary across different operating systems or NAMD versions, but if two decimal places are retained, the results will be identical or very similar.

Part III. Decomposing the free energy contributions of a per-residue

3.2. Decomposing the free energy

If the users want to use MolAICal to decompose the free energy contributions of a per-residue, it can draw lessons from the above steps. For example, the residue M165 of SARS-CoV-2 Mpro is reported to interact with ligand N3. So the residue M165 is chosen as an example. First of all, change the console into the "004-MMGBSA\Decompose" folder:

#> cd 004-MMGBSA\Decompose

3.2.1. Extracting trajectory of an appointed residue in complex with ligand

#> molaical.exe -call run -n vmd -c stripdcd -i -dispdev text -psf "../mpro.psf" -args protein,and,resid,165,or,resname,LIG "../mpro.dcd" "res-lig" ../mpro.psf ../mpro.pdb

-i:	after "-i", follow with VMD-related commands. The command-line argument "-i" **must** be placed **after** other arguments (e.g., after argument "-c").
-args:	it is the usage like the command "**atomselect**" of VMD software, such as "atomselect top protein and resid 165 or resname LIG". Here, the comma "," represents blank space " ".
-call:	when its value is "run", it will call the external program.
-n:	the external program name, it will automatically search the path of the program based on the name of this program from the inner environment variables of MolAICal.
-c:	the calculation way, it will be used to deal with the DCD file if its value is "stripDCD".

"res-lig" is the prefix of the output files. The mpro.psf and mpro.pdb are referred to generate the files that have the prefix "res-lig". It will generate res-lig.psf, res-lig.pdb, and res-lig.dcd. Open "res-lig.conf" and modify the appropriate parameters in a similar way to "complex.conf".

Our tutorial is run by CPU. Users can run it on GPUs. Running NAMD command in Linux operating system as below:

#> molaical.exe -call run -n namd -i +p1 res-lig.conf >& res-lig.log &

3.2.2. Extracting trajectory of an appointed residue only

#> molaical.exe -call run -n vmd -c stripdcd -i -dispdev text -psf "../mpro.psf" -args protein,and,resid,165 "../mpro.dcd" "res" ../mpro.psf ../mpro.pdb

"res" is the prefix of the output files. The mpro.psf and mpro.pdb are referred to generate the files that have the prefix "res". It will generate res.psf, res.pdb, and res.dcd. Open "res.conf" and modify

the appropriate parameters in a similar way to “complex.conf”.

Our tutorial is run by CPU. Users can run it on GPUs. Running NAMD command in Linux operating system as below:

#> molaical.exe -call run -n namd -i +p1 res.conf >& res.log &

3.2.3. Extracting trajectory of ligand only

#> molaical.exe -call run -n vmd -c stripdcd -i -dispdev text -psf "../mpro.psf" -args resname,LIG "../mpro.dcd" "lig" ../mpro.psf ../mpro.pdb

"lig" is the prefix of the output files. The mpro.psf and mpro.pdb are referred to generate the files that have the prefix "lig". It will generate lig.psf, lig.pdb, and lig.dcd. Open “lig.conf” and modify the appropriate parameters in a similar way to “complex.conf”.

Our tutorial is run by CPU. Users can run it on GPUs. Running NAMD command in Linux operating system as below:
#> molaical.exe -call run -n namd -i +p1 lig.conf >& lig.log &

3.2.4. Calculating free energy contributions of a per-residue by MolAICal

#> molaical.exe -mmgbsa -c res-lig.log -r res.log -l lig.log

The output contains the binding free energy of a per-residue △G as follows:

---

delta E(internal): -0
delta E(electrostatic) + deltaG(sol): -1.4609
delta E(VDW): -5.3395
delta G binding: -6.8004 +/- 1.1236 (kcal/mol)

Note: the users can use the above similar method to calculate the free energy contribution of pairwise and per-residue by MolAICal. Results may vary across different operating systems or NAMD versions, but if two decimal places are retained, the results will be identical or very similar.

Part IV. Entropy calculation

3.3. Entropy calculation by Carma and MolAICal

3.3.1. Download the Carma

Carma can be run on Windows or Linux operating systems. The new version of Carma can be downloaded from https://github.com/glykos/carma, https://github.com/glykos/carma, http://utopia.duth.gr/~glykos/progs, or http://utopia.duth.gr/~glykos.

I recommend the latest version of Carma for entropy calculation. The previous version will show the “NaN” errors, etc. Please check the issue report below:
https://groups.google.com/g/carma-molecular-dynamics/c/KpyY5sEkrj4

Here, 25 frames are extracted from DCD trajectories for entropy calculation. The users can extract a suitable number of frames for their projects. More number of frames will cost more running time.

According to the LICENSE of Carma (https://github.com/glykos/carma/blob/master/LICENSE), which is without the limitation rights to use, copy, modify, merge, publish, distribute, etc. Hence, MolAICal can directly integrate and invoke Carma to fit the molecules and calculate the entropy of molecules **without downloading Carma by users again.**

3.3.2. Calculate the complex entropy

#> cd 004-MMGBSA\entropy

Go to the folder “com”. This step is for removing rotations and translations.

#> molaical.exe -call run -c entropy -i -v -fit -force -atmid ALLID -segid A -segid C complex.dcd complex.psf

-call:	when its value is "run", it will call the external program.
-c:	the calculation way, it will be used to calculate entropy by Carma if its value is "entropy".
-i:	after "-i", follow with Carma-related commands. The command-line argument "-i" **must** be placed **after** other arguments (e.g., after argument "-c").
-v:	verbose information
-fit:	Remove global rotations-translations from DCD frames
-force:	tells Carma to stop the calculation of entropies before they become undefined.
-atmid:	atom-name-based atom selection
-segid:	segment-identifier-based atom selection (dcd-psf only)

This step is for calculating entropy.

#> molaical.exe -call run -c entropy -i -v -cov -eigen -mass -force -temp 310 -atmid ALLID -segid A -segid C -last 25 carma.fitted.dcd complex.psf >& com.log &

-call:	when its value is "run", it will call the external program.
-c:	the calculation way, it will be used to calculate entropy by Carma if its value is "entropy".
-i:	after "-i", follow with Carma-related commands. The command-line argument "-i" **must** be placed **after** other arguments (e.g., after argument "-c").
-v:	verbose information
-cov:	calculate variance-covariance matrix
-eigen:	calculate eigenvectors and values of covariance matrices
-mass:	use mass-weighting for the covariance matrix
-force:	tells carma to stop the calculation of entropies before they become undefined.
-atmid:	atom-name-based atom selection
-segid:	segment-identifier-based atom selection (dcd-psf only)

3.3.3. Calculate the receptor entropy

Go to the folder “rec”. This step is for removing rotations and translations.

#> molaical.exe -call run -c entropy -i -v -fit -atmid ALLID -segid A protein.dcd protein.psf

This step is for calculating entropy.

#> molaical.exe -call run -c entropy -i -v -cov -eigen -mass -force -temp 310 -atmid ALLID -segid A -last 25 carma.fitted.dcd protein.psf >& rec.log &

3.3.4. Calculate the ligand entropy

Go to the folder “lig”. This step is for removing rotations and translations.

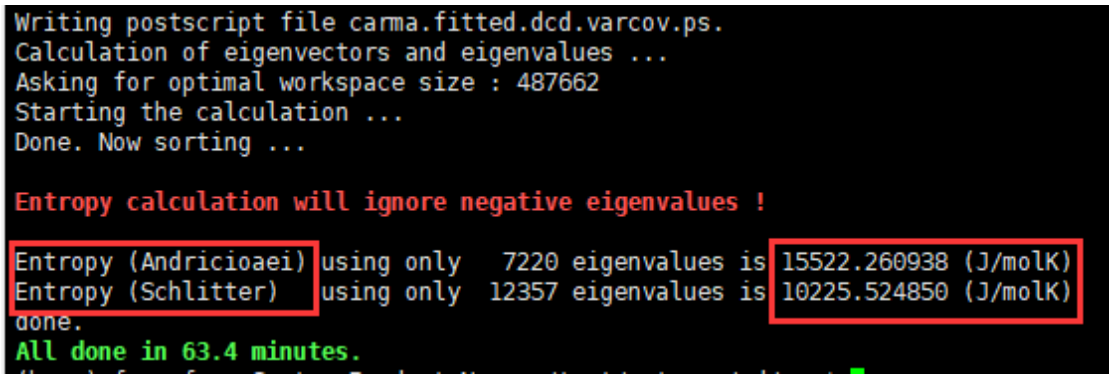
#> molaical.exe -call run -c entropy -i -v -fit -force -atmid ALLID -segid C ligand.dcd ligand.psf

This step is for calculating entropy.

#> molaical.exe -call run -c entropy -i -v -cov -eigen -mass -force -temp 310 -atmid ALLID -segid C -last 25 carma.fitted.dcd ligand.psf >& lig.log &

3.3.5. Calculate temperature multiplied by the delta entropy

Please check log files: “com.log”, “rec.log”, and “lig.log”, and find the Andricioaei’s or Schlitter’s entropy values in the files of “com.log”, “rec.log”, and “lig.log”. For example, it can find the Andricioaei’s or Schlitter’s entropy value in the content of “com.log” (see the below figure):



Change to the parent directory, run MolAICal based on log files: “com.log”, “rec.log”, and “lig.log” as follows:

#> molaical.exe -entropy -if -c com/com.log -r rec/rec.log -l lig/lig.log -t 310.0

It will show the result:

Entropy Type: Andricioaei

The entropy (T * delta S) = -25.3034 (kcal/mol)

Entropy Type: Schlitter

The entropy (T * delta S) = -32.3195 (kcal/mol)

Where: **-entropy:** means delta entropy calculation

-c: the value or file of complex entropy. Here, it is a result file.

-r: the value or file of the receptor or first molecule entropy. Here, it is a result file.

-l: the value or file of the ligand or second molecule entropy. Here, it is a result file.

-t: the Kelvin temperature in MD simulation.

if: it will read entropy values from result files, if the input arguments contain "-if".

Note: Calculating entropy under Windows operating system is very slow, possibly only reaching Schlitter's entropy value; it is recommended to perform entropy calculations on a Linux operating system instead.

If entropy is considered, the MM/GBSA is estimated by the following equations:

$\Delta G_{bind} = \Delta H - T\Delta S \approx \Delta E_{MM} + \Delta G_{sol} - T\Delta S$

$\Delta E_{MM} = \Delta E_{internal} + \Delta E_{ele} + \Delta E_{vdw}$

$\Delta G_{sol} = \Delta G_{GB} + \Delta G_{SA}$

As mentioned above, the energy value of entropy was not considered, as follows:

delta G binding: -43.0921 +/- 1.5748 (kcal/mol)

Using Entropy Type: Andricioaei

ΔG_{bind} = -43.0921 – (-25.3034) = -17.7887 (kcal/mol)

Using Entropy Type: Schlitter

ΔG_{bind} = -43.0921 – (-32.3195) = -10.7726 (kcal/mol)

Note: If there are no binding-induced structural changes in the process of molecular dynamics (MD) simulations, or very similar structural changes among different systems, the entropy calculation can be omitted. Using only the △G binding value without considering entropy is sufficient.

Disscusions

How to understand that more calculated frames lead to greater entropy? The following is merely a personal perspective:

One can envision it like this: Even within a fitted trajectory, consider just a single CA atom. It can undergo random motion within a certain range. The more frames you have, the higher the degree of disorder observed, and thus the more likely the entropy value is higher. Therefore, calculating more frames makes the apparent disorder greater, increasing the likelihood of a higher entropy value.

Unless the calculation reveals ordered patterns, such as symmetry, which might be difficult to observe within the limited simulation time.

So, how to determine the number of frames to use for entropy when evaluating binding energy? Two approaches can be explored:

Option 1: When systems are at equilibrium, for comparing multiple systems (e.g., comparing binding energies of multiple drugs in the receptor pocket), the frames can be extracted from the trajectories at regular intervals. For example, if a 500 ns simulation was run, take 50 frames, meaning one frame every 10 ns. Apply this same sampling strategy to all systems being compared when calculating and comparing binding free energies.

Option 2: When a system is at equilibrium: as many researchers do, select a specific set of frames of interest for study. For instance, analyze the last 50 frames of the trajectory to investigate the impact of entropy on the binding free energy.

Appendix 1

If users want to understand the detailed operation process of MM/GBSA, they can repeat the above tutorial. Meanwhile, MolAICal offers a fast and simple calculation method for MM/GBSA that does not require the complex steps mentioned above.

Assuming users have configured the internal commands in MolAICal for VMD and NAMD using the following instructions:

```
#> molaical.exe -call set -n VMD -p "C:\Program Files (x86)\University of Illinois\VMD\vmd.exe"
#> molaical.exe -call set -n namd -p "d:\namd2\namd2.exe"
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system. When running MMPB/GB/SA functionality, and setting environment variables using MolAICal (e.g., #> molaical.exe -call set -n VMD -p "path"), the variable names in MMPB/GB/SA corresponding to '-n' are fixed: 'vmd', 'namd', and 'delphi' (case insensitive).

✧ **Switch to the "fast" directory and run the following command:**

#> molaical.exe -mmpbgbsa -f mmgbsa.vmd

It can modify the arguments in the file 'mmgbsa.vmd', or modify the arguments by inputting '-i' argument. For more detailed information, please check the MolAICal manual.

The output results from the command (#> molaical.exe -mmpbgbsa -f mmgbsa.vmd) have the same value as the above results: **delta G binding:** -43.0921 +/- 1.5748 (kcal/mol)

For the file 'mmgbsa_create_file.vmd', it has more arguments "-namd_config_file input.namd \" than 'mmgbsa.vmd'. This allows users to customize input parameters of NAMD in the file "input.namd". If running the command below:

#> molaical.exe -mmpbgbsa -f mmgbsa_create_file.vmd

It will have the same result as the above tutorial results.

✧ **Switch to the "Decompose" directory for decomposing the free energy contributions of a per-residue. It has the same run way as the above command:**

#> molaical.exe -mmpbgbsa -f mmgbsa.vmd

or

#> molaical.exe -mmpbgbsa -f mmgbsa_create_file.vmd

It will have the same result as the above tutorial results. It seems very quick and easy from this way.

✧ **Switch to the "entropy" directory for the entropy calculation. MolAICal also provides a simple way for the entropy calculation as follows:**

```
#> molaical.exe -call run -c script -n runscript -i com rec lig
```

The strings after '-i' are the parameter list that will be split by space. The split strings will replace the strings $molargs1, $molargs2, $molargs3, etc. in the script file. The number of the string $molargs1, $molargs2, $molargs3, etc., is the same as the length of the split strings after '-i'. The **'-n'** assigns the name of the script file named "runscript":

```
# 1. calculate complex part (rename in win: move /y carma.fitted.dcd complex_fit.dcd)
cd $molargs1
molaical.exe -call run -c entropy -i -v -fit -force -atmid ALLID -segid A -segid C complex.dcd complex.psf
molaical.exe -call run -c entropy -i -v -cov -eigen -mass -force -temp 310 -atmid ALLID -segid A -segid C -last 25 carma.fitted.dcd complex.psf > com.log
cd ..

# 2. calculate receptor part
cd $molargs2
molaical.exe -call run -c entropy -i -v -fit -atmid ALLID -segid A protein.dcd protein.psf
molaical.exe -call run -c entropy -i -v -cov -eigen -mass -force -temp 310 -atmid ALLID -segid A -last 25 carma.fitted.dcd protein.psf > rec.log
cd ..

# 3. calculate receptor part
cd $molargs3
molaical.exe -call run -c entropy -i -v -fit -force -atmid ALLID -segid C ligand.dcd ligand.psf
molaical.exe -call run -c entropy -i -v -cov -eigen -mass -force -temp 310 -atmid ALLID -segid C -last 25 carma.fitted.dcd ligand.psf > lig.log
cd ..

# in windows: | findstr /I /R "entropy" > results.log      in linux: | grep -i "entropy" > results.log
molaical.exe -entropy -if -c $molargs1/com.log -r $molargs2/rec.log -l $molargs3/lig.log -t 310.0 > results.log
```

Note: "molaical.exe" does not need to be modified; MolAICal will automatically replace its path for "molaical.exe".

The detailed arguments for 'mmgbsa.vmd' or 'mmgbsa_create_file.vmd' are as follows:

usage: mmpbgbsa	
-f <arg>	The path of the input file for MM/PB/GB/SA calculation.
-h	Print help information.
-i	The command arguments of the program mmpbgbsa. It includes all command-line arguments of external programs, but the '-i' parameter must be the last one specified, while all other identifiers (e.g., '-f', etc.) must come before '-i'.
-mmpbgbsa	MM/PB/GBSA calculation.
-n <arg>	The name of the executed program VMD, the default value is 'vmd'.
-o <arg>	Whether to print the output of the program.

-p <arg>	The path of the program VMD. The MolAICal environment setup command can be used to configure it once, eliminating the need for tedious path input later.

Usage: -mmgbpbsa -f xxx.vmd -i -system_topology top_file -system_coords coords_file -trajectory_files filename [-args...]

Mandatory arguments:

-system_topology	<topology filename>
-trajectory_files	<trajectory filename>
-system_coords	<coordinates filename>

Optional arguments:

-namd_config_file	<namd custom configuration file>
-pb_gb_file	<PB or GB custom configuration file>
-pb_gb_temperature	<temperature> -- default: 310K
-topology_type	<topology filetype> -- default: auto
-trj_type	<trajectory filetype> -- default: auto
-force_parameters	<force field parameters> -- default: auto-detect
-output_filename	<output filename> -- default: mmgbpbsa.log
-debug	<debug level> -- default: 0. It can be 0, 1, 2. If '2' is selected, it will save all generated temporary files. If '1' is selected, it will save some of the generated temporary files. If '0' is selected, it will only save the result files.
-first_frame	<first frame> -- default: 0
-last_frame	<last frame> -- default: -1
-frame_stride	<stride> -- default: 1
-complex_selection	<complex selection> -- default: ""
-receptor_selection	<receptor selection> -- default: ""
-ligand_selection	<ligand selection> -- default: ""
-molecular_mechanics	<do gas-phase calc> -- default: 0
-namd_executable	<path to NAMD> -- default: "namd2"
-namd_cutoff	<cutoff for MD simulations> -- default: 16.0
-namd_switching	<"on" means smooth switching function truncates van der Waals potential energy at the cutoff distance. "off" will close smooth switching function> -- default: on
-namd_switchdist	<distance at which to activate switching function> -- default: "namd2"
-mm_dielectric	<dielectric constant> -- default: 1.0
-poisson_boltzmann	<do PB calculation> -- default: 2. It can be '0, 1, 2'. If '0' is selected, it will calculate MM/GBSA. If '1' is selected, it will calculate MM/PBSA with the program Delphi. If '2' is selected, it will calculate MM/PBSA with the program APBS.
-pb_executable	<path to DelPhi/APBS> -- default: "apbs"
-pb_radii_type	<type of PB radii> -- default: bondi
-pb_boundary_flag	<boundary condition> -- default: sdh
-pb_charge_method	<charge method> -- default: spl0
-ion_charge_apbs_negtive	<total anionic charge> -- default: -1.0
-ion_charge_apbs_positive	<total cationic charge> -- default: 1.0

-ion_apbs_radius	<mobile ion species radius> -- default: spl0
-generalized_born	<do GB calculation> -- default: 0
-gb_exterior_dielectric	<external dielectric> -- default: 78.5
-pb_gb_ion_concentration	<ion concentration> -- default: 0.15
-gb_surface_area	<do LCPO calculation> -- default: 0
-gb_surface_gamma	<surface tension> -- default: 0.0072
-surface_accessibility	<do SA calculation> -- default: 1, it can be 1 or 2, "if '1', SASA calculation is performed by VMD; if '2', SASA calculation is performed by APBS.
-sa_input_apbs	<APBS input file for SA calculation> -- default: ""
-sa_radii_type	<type of SA radii> -- default: bondi
-sa_gamma	<surface tension> -- default: 0.0072
-sa_beta	<surface offset> -- default: 0.0
-sa_probe_radius	<radius of probe> -- default: 1.4
-sa_displacement_apbs	<displacement parameter for finite-difference-based force calculations (surface area derivatives), with unit: Å.> -- default: 0.2
-sa_sample_points	<number of samples> -- default: 500
-namd_seed	<namd seed for reproducibility> -- default: 12345
-num_calc_cores	<number of CPU cores for calculations> -- default: 1

Appendix 2

Install external programs inside the MolAICal container (if finished, only for Linux Version MolAICal)

Please note that the configuration of MolAICal differs between the Windows and Linux versions: the Linux version utilizes the udocker container approach and requires setup within the container, whereas the Windows version does not.

To address the library dependency issues in Linux, the Linux version of MolAICal adopts a container-based approach. If **external programs need** to be invoked, it is **recommended** to install them **within** the MolAICal **container**. If **no external** programs are required, this step can be **ignored**. This tutorial requires the external programs NAMD and VMD, and the steps are as follows:

a) First of all, copy the files to the MolAICal container.

```
****** 1.  Go to the container file system, it will enter the "/root" ******
#> molaical.exe -eset shell in


****** 2.  The first part of 'cp' is from the local machine, and the second part is in the container; ******
```

```
****** The VMD and NAMD packages can be moved into the container by the command below ******
#> cp /home/user/<local machine file> /root/soft

****** 3.  Exit container file system ******
#> exit
```

b) Secondly, enter the container virtual environment of MolAICal:

```
#> molaical.exe -eset sys run molaical
```

Note: 'molaical' is the container name.

c) Installing software in the container virtual environment of MolAICal is the same as installing it on the host local machine. Here, we take the installation of VMD and NAMD as examples:

1) For NAMD:

Extract the NAMD files, assuming the extracted folder is named namdcpu. Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n NAMD -p "/root/soft/namdcpu/namd3"
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system. The string "VMD" and "NAMD" (case insensitive) after "-n" is fixed for the paths of VMD and NAMD. To ensure the reproducibility of MM/GBSA results, it is recommended to use the CPU version of NAMD, as the "seed" parameter appears to be ineffective for reproducibility in the CUDA version of NAMD.

2) For VMD:

Following the steps below:

✧ Extract the VMD files:

```
#> tar -xzvf vmd-xxx.tar.gz
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system.

✧ Modify the install path in a file named "**configure**" in the decompressed folder of VMD:

```
The default: $install_bin_dir="/usr/local/bin"; modify it to →
$install_bin_dir="/root/soft/vmd193/bin";

The default: $install_library_dir="/usr/local/lib/$install_name"; modify it to →
$install_library_dir="/root/soft/vmd193/lib/$install_name";
```

✧ Install VMD:

```
#> cd vmd-xx
#> ./configure LINUXAMD64
#> cd src
#> make install
```

Note: Please run **./configure**, and select the right types for the used computers. Here, it is "LINUXAMD64".

✧ Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n VMD -p "/root/soft/vmd193/bin/vmd"
```

So far, all the installations and configurations of NAMD and VMD are complete in the MolAICal container.

3) Save and load the modified container (Option)

To preserve changes made within a container and avoid reinstalling software later, since all data will be lost if the container is deleted.

◆ The first step is to obtain the container ID and name:

```
#> molaical.exe -eset sys ps
```

◆ Secondly, clone this container as follows:

```
#> molaical.exe -eset sys export --clone -o molaicalv2.tar molaical
```

Note: 'molaical' is the container name. It can also be the container ID. If an error is reported, it may be due to permission issues with the mounted file system, which can be ignored.

◆ Thirdly, import the cloned container **if needing**:

```
#> molaical.exe -eset sys import --clone --name molaicalv2 molaicalv2.tar
```

◆ Now, change the loaded new container name to the default container name "molaical", the previous container is renamed to "bakmolaical", the commands are as follows:

```
#> molaical.exe -eset sys rename molaical bakmolaical
#>molaical.exe -eset sys rename molaicalv2 molaical
```

Notice: Please note that a **container** (**dynamic**) is generated from an **image** (**static**). Operations such as software installation must be performed within the dynamic container; if cloning this container, restoring it will still be based on the original underlying image.

For more information, please see the manual of MolAICal or run "molaical.exe -eset sys --help" to get more help details.